청약 전략 플랫폼 — 프로젝트 폴더 구조

2026년 9월 · 현재 스캐폴딩된 backend / frontend 구조 정리

```
cheongyak-platform/
├── README.md                          실행 방법, 진행 체크리스트
├── .gitignore
├── docker-compose.yml                 postgres + backend + frontend 한 번에 실행
├── docs/
│   └── 청약_전략_플랫폼_기획안.pdf    팀 공유용 기획안
│
├── backend/                           FastAPI 서버
│   ├── requirements.txt
│   ├── Dockerfile
│   ├── .env.example
│   └── app/
│       ├── main.py                    엔트리포인트, 라우터 등록, /health
│       ├── core/
│       │   └── config.py              환경변수 설정
│       ├── db/
│       │   ├── base.py                SQLAlchemy Base
│       │   └── session.py             DB 세션
│       ├── models/                    DB 테이블 정의
│       │   ├── user.py                UserProfile
│       │   ├── listing.py             Listing (청약 공고)
│       │   ├── historical_result.py 과거 경쟁률·커트라인
│       │   └── loan_product.py        LoanProduct
│       ├── schemas/                   요청/응답 형식
│       │   ├── user.py
│       │   ├── listing.py
│       │   ├── loan.py
│       │   └── recommendation.py
│       ├── api/routes/                엔드포인트
│       │   ├── recommendations.py     GET /recommendations/{user_id}
│       │   ├── listings.py            GET /listings, /listings/{id}
│       │   ├── loans.py               POST /loans/match
│       │   └── chat.py                POST /chat (RAG 자리, 미연동)
│       ├── services/                  매칭 엔진 (핵심 로직)
│       │   ├── eligibility_filter.py      1단계 자격필터+가점계산
│       │   ├── probability_estimator.py   2단계 확률(구간) 추정
│       │   └── loan_matcher.py            3단계 대출 매칭
│       └── data_pipeline/             외부 데이터 수집
│           ├── applyhome_client.py    청약홈 API 연동 (TODO)
│           └── scheduler.py           주기적 수집 스케줄러
│   └── tests/
│       └── test_health.py
│
└── frontend/                          React + Redux
    ├── package.json
    ├── Dockerfile
    ├── .env.example
    ├── public/
    │   └── index.html
    └── src/
        ├── index.js                   진입점 (Provider+Router)
        ├── App.js                     라우트 4개 정의
        ├── components/
        │   └── Layout.jsx             공통 레이아웃/네비게이션
        ├── pages/                     화면 4종
        │   ├── InputFormPage.jsx        ① 내 정보 입력
        │   ├── DashboardPage.jsx        ② 추천 대시보드
        │   ├── ListingDetailPage.jsx    ③ 공고 상세
        │   └── LoanSimulatorPage.jsx    ④ 대출 시뮬레이션
        ├── store/                     Redux Toolkit
        │   ├── index.js
        │   ├── userSlice.js
        │   ├── listingsSlice.js
        │   └── loansSlice.js
        ├── api/
        │   └── client.js              axios 인스턴스
        └── styles/
            └── global.css
```

폴더별 역할 요약

폴더	역할
`docs/`	팀 공유용 기획 문서 보관
`backend/app/models/`	기획안 5장 데이터 구성(청약공고·과거경쟁률·대출상품·사용자프로필)을 그대로 테이블로 정의
`backend/app/services/`	기획안 7장 매칭 엔진 3단계(자격필터 → 확률추정 → 대출매칭) 로직이 들어가는 곳 — 지금은 TODO 위주
`backend/app/api/routes/`	프론트엔드가 호출할 API 엔드포인트
`backend/app/data_pipeline/`	청약홈 공공데이터 API 등 외부 데이터를 가져오는 수집기
`frontend/src/pages/`	기획안 4장 화면 흐름(입력폼→대시보드→상세→대출시뮬레이션) 4개 화면
`frontend/src/store/`	화면 간 공유되는 상태(사용자 입력값, 추천 결과, 대출 매칭 결과)

지금 채워야 할 우선순위 (로드맵 1단계)

파일	할 일
`services/eligibility_filter.py`	지금은 항상 통과하는 자리표시자 — 무주택기간/지역/소득 기준 실제 조건 구현
`data_pipeline/applyhome_client.py`	data.go.kr에서 발급받은 실제 API 키·엔드포인트로 교체
`models/*.py`	Alembic으로 마이그레이션 파일 생성 후 실제 DB에 테이블 생성
`pages/InputFormPage.jsx`	제출 시 백엔드에 사용자 저장하는 API 연동 (현재는 바로 대시보드로 이동만 함)